

AFYA UNIVERSIDADE UNIGRANRIO

Curso: Análise e Desenvolvimento de Sistemas

INGRID RODRIGUES DE ARAUJO

ATIVIDADE 1 – ESTRUTURAS DE DADOS

Recursividade em Python

DUQUE DE CAXIAS - RJ

2026

INGRID RODRIGUES DE ARAUJO

ATIVIDADE 1 – ESTRUTURAS DE DADOS

Recursividade em Python

Trabalho apresentado à disciplina de Estruturas de Dados como requisito de avaliação da Atividade 1. Professor: Carlos Ribeiro da Cunha.

DUQUE DE CAXIAS - RJ

2026

SUMÁRIO

1 INTRODUÇÃO	4
2 FUNÇÃO FATORIAL	5
3 FUNÇÃO FIBONACCI	6
4 FUNÇÃO SOMALISTA	7
5 FUNÇÃO PALÍNDROMO	9
6 FUNÇÃO CONTAOCORRÊNCIA	10
7 CONCLUSÃO	12
REFERÊNCIAS	13

1 INTRODUÇÃO

A programação permite criar diferentes maneiras de resolver um mesmo problema. Uma das estratégias usadas para isso é a recursividade. Nessa solução, uma função chama a si mesma e trabalha com partes menores do problema até chegar a uma situação mais simples, chamada caso base. Para que a recursão funcione bem, essas chamadas precisam se aproximar desse caso base. Isso evita que o processo continue sem parar (Carvalho, 2024).

Segundo Carvalho (2024), as soluções recursivas podem deixar alguns algoritmos mais fáceis de criar e entender. No entanto, dependendo do problema, elas podem gastar mais tempo de processamento e outros recursos quando comparadas com as soluções iterativas. Entre os exemplos mostrados no material da disciplina estão o cálculo do fatorial e a sequência de Fibonacci. Esses exemplos ajudam a ver, na prática, como uma função pode usar resultados de chamadas anteriores para chegar à solução desejada.

Com base nesses conceitos, esta atividade tem o objetivo de aplicar recursividade ao criar diferentes programas em Python. Para isso, serão desenvolvidas funções para calcular o fatorial de um número, encontrar valores da sequência de Fibonacci, somar os elementos de uma lista, verificar se uma palavra é um palíndromo e contar quantas vezes um elemento aparece em um vetor. Junto com os códigos, serão apresentados exemplos de uso, com os resultados e as telas de execução dos programas. Assim, será possível ver na prática como cada solução funciona.

2 FUNÇÃO FATORIAL

Neste programa, utilizei a recursividade para calcular o fatorial de um número. Quando n é igual a zero, a função retorna 1, encerrando as chamadas recursivas. Nos outros casos, o número é multiplicado pelo fatorial do número anterior. Essa lógica continua até chegar ao caso base.

```
01_fatorial.py > ...
1  # =====
2  # Calcular o fatorial de forma recursiva
3  # =====
4
5  def fatorial(n):
6      if n < 0:
7          return None          # número negativo não tem fatorial
8      elif n == 0:
9          return 1             # caso base: 0! = 1
10     else:
11         return n * fatorial(n - 1)  # caso recursivo: n! = n * (n-1)!
12
13
14 print("Fatorial(-2) =", fatorial(-2))
15 print("Fatorial(0) =", fatorial(0))
16 print("Fatorial(1) =", fatorial(1))
17 print("Fatorial(5) =", fatorial(5))
18 print("Fatorial(7) =", fatorial(7))
19
```

Print 1 – Programa Fatorial

Exemplos de utilização

Para verificar se a função funciona, utilizei valores diferentes, incluindo um número negativo, os casos iniciais e números maiores.

```
print("Fatorial(-2) =", fatorial(-2))
print("Fatorial(0) =", fatorial(0))
print("Fatorial(1) =", fatorial(1))
print("Fatorial(5) =", fatorial(5))
print("Fatorial(7) =", fatorial(7))
```

Resultados obtidos

Após executar o programa, foram obtidos os seguintes resultados:

```
01_fatorial.py > ...
1 # =====
2 # Calcular o fatorial de forma recursiva
3 # =====
4
5 def fatorial(n):
6     if n < 0:
7         return None          # número negativo não tem fatorial
8     elif n == 0:
9         return 1             # caso base: 0! = 1
10    else:
11        return n * fatorial(n - 1) # caso recursivo: n! = n * (n-1)!
12
13
14 print("Fatorial(-2) =", fatorial(-2))
15 print("Fatorial(0) =", fatorial(0))
16 print("Fatorial(1) =", fatorial(1))
17 print("Fatorial(5) =", fatorial(5))
18 print("Fatorial(7) =", fatorial(7))
19

```

Problemas Saída Console De Depuração **Terminal** Portas

```
PS C:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py> python -u "c:\Users\Ingrid\Documents\Códigos\recursividade-
codigos-py\01_fatorial.py"
Fatorial(-2) = None
Fatorial(0) = 1
Fatorial(1) = 1
Fatorial(5) = 120
Fatorial(7) = 5040
PS C:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py>
```

Ln 20, Col 1 Espaços: 4 UTF-8 CRLF Python Python 3.14.7

Print 2 – Execução do Programa Fatorial

3 FUNÇÃO FIBONACCI

Nesta questão, utilizei novamente a recursividade. Para valores menores que zero, a função retorna 0. Para 0 e 1, retorna 1, conforme diz no enunciado. Para os demais valores, o resultado é a soma dos dois termos anteriores.

```
02_fibonacci.py > ...
1 # =====
2 # Implementar F(n) de acordo com a definição por partes
3 # =====
4
5 def F(n):
6     if n < 0:
7         return 0             # caso especial: n negativo
8     elif n == 0:
9         return 1             # caso base 1
10    elif n == 1:
11        return 1             # caso base 2
12    else:
13        return F(n - 1) + F(n - 2) # caso recursivo
14
15
16 print("F(-2) =", F(-2))
17 print("F(0) =", F(0))
18 print("F(1) =", F(1))
19 print("F(2) =", F(2))
20 print("F(5) =", F(5))
21 print("F(6) =", F(6))
22

```

Print 3 – Programa Fibonacci

Exemplos de utilização

Escolhi valores diferentes para ver como a função se comporta nos casos definidos diretamente. Também testei nos valores que precisam de chamadas recursivas.

```
print("F(-2) =", F(-2))
print("F(0) =", F(0))
print("F(1) =", F(1))
print("F(2) =", F(2))
print("F(5) =", F(5))
print("F(6) =", F(6))
```

Resultados obtidos

Após executar o programa, foram obtidos os seguintes resultados:



```
02_fibonacci.py > ...
1 # =====
2 # Implementar F(n) de acordo com a definição por partes
3 # =====
4
5 def F(n):
6     if n < 0:
7         return 0                # caso especial: n negativo
8     elif n == 0:
9         return 1                # caso base 1
10    elif n == 1:
11        return 1                # caso base 2
12    else:
13        return F(n - 1) + F(n - 2) # caso recursivo
14
15
16 print("F(-2) =", F(-2))
17 print("F(0) =", F(0))
18 print("F(1) =", F(1))
19 print("F(2) =", F(2))
20
Problemas Saída Console De Depuração Terminal Portas
codigos-py\02_fibonacci.py
F(-2) = 0
F(0) = 1
F(1) = 1
F(2) = 2
F(5) = 8
F(6) = 13
PS C:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py>
Ln 23, Col 1 Espaços: 4 UTF-8 CRLF Python Python 3.14.7
```

Print 4 – Execução do Programa Fibonacci

4 FUNÇÃO SOMALISTA

Para realizar a soma, considerei a lista vazia como o caso base, retornando 0. Se ainda houver elementos na lista, a função pega o primeiro valor e soma com o

resultado da chamada para o resto da lista. Assim, a lista fica menor a cada chamada até ficar vazia.

```
03_somalista.py > ...
1  # =====
2  # Implementar SomaLista(lista) de forma recursiva
3  # =====
4
5  def SomaLista(lista):
6      if len(lista) == 0:
7          return 0                                # caso base: lista vazia, soma é 0
8      else:
9          return lista[0] + SomaLista(lista[1:]) # caso recursivo: primeiro elemento + soma do resto
10
11
12 print("Soma [1, 3, 5, 7, 9] =", SomaLista([1, 3, 5, 7, 9]))
13 print("Soma [2, 4, 6] =", SomaLista([2, 4, 6]))
14 print("Soma [10, 20, 30] =", SomaLista([10, 20, 30]))
15 print("Soma [] =", SomaLista([]))
16
```

Print 5 – Programa SomaLista

Exemplos de utilização

Além da lista apresentada no enunciado, testei outras listas para verificar se a função ainda funcionava com quantidades e valores diferentes. Também testei uma lista vazia para verificar o caso base.

```
print("Soma [1, 3, 5, 7, 9] =", SomaLista([1, 3, 5, 7, 9]))
print("Soma [2, 4, 6] =", SomaLista([2, 4, 6]))
print("Soma [10, 20, 30] =", SomaLista([10, 20, 30]))
print("Soma [] =", SomaLista([]))
```

Resultados obtidos

Após executar o programa, foram obtidos os seguintes resultados:

```
03_somalista.py > ...
1 # =====
2 # Implementar SomaLista(lista) de forma recursiva
3 # =====
4
5 def SomaLista(lista):
6     if len(lista) == 0:
7         return 0 # caso base: lista vazia, soma é 0
8     else:
9         return lista[0] + SomaLista(lista[1:]) # caso recursivo: primeiro elemento + soma do resto
10
11
12 print("Soma [1, 3, 5, 7, 9] =", SomaLista([1, 3, 5, 7, 9]))
13 print("Soma [2, 4, 6] =", SomaLista([2, 4, 6]))
14 print("Soma [10, 20, 30] =", SomaLista([10, 20, 30]))
15 print("Soma [] =", SomaLista([]))
16
17 |
```

```
Problemas Saída Console De Depuração Terminal Portas
PS C:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py> python -u "c:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py\03_somalista.py"
Soma [1, 3, 5, 7, 9] = 25
Soma [2, 4, 6] = 12
Soma [10, 20, 30] = 60
Soma [] = 0
PS C:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py>
```

Print 6 – Execução do Programa SomaLista

5 FUNÇÃO PALÍNDROMO

Para saber se uma palavra é um palíndromo, comparei a primeira letra com a última. Se forem diferentes, a função retorna False. Se forem iguais, essas duas letras são retiradas da próxima verificação e a função analisa o restante da palavra. Quando sobra apenas uma letra ou nenhuma, significa que todas as comparações deram certo e a função retorna True.

```
04_palindromo.py > ...
1 # =====
2 # Implementar Palindromo(palavra) de forma recursiva
3 # =====
4
5 def Palindromo(palavra):
6     palavra = palavra.lower() # padroniza para minúsculas
7     if len(palavra) <= 1:
8         return True # caso base: 0 ou 1 letra é sempre palíndromo
9     elif palavra[0] != palavra[-1]:
10         return False # caso especial: extremos diferentes
11     else:
12         return Palindromo(palavra[1:-1]) # caso recursivo: remove as pontas e repete
13
14
15 print("arara:", Palindromo("arara"))
16 print("radar:", Palindromo("radar"))
17 print("ovo:", Palindromo("ovo"))
18 print("python:", Palindromo("python"))
19 print("casa:", Palindromo("casa"))
20
```

Print 7 – Programa Palíndromo

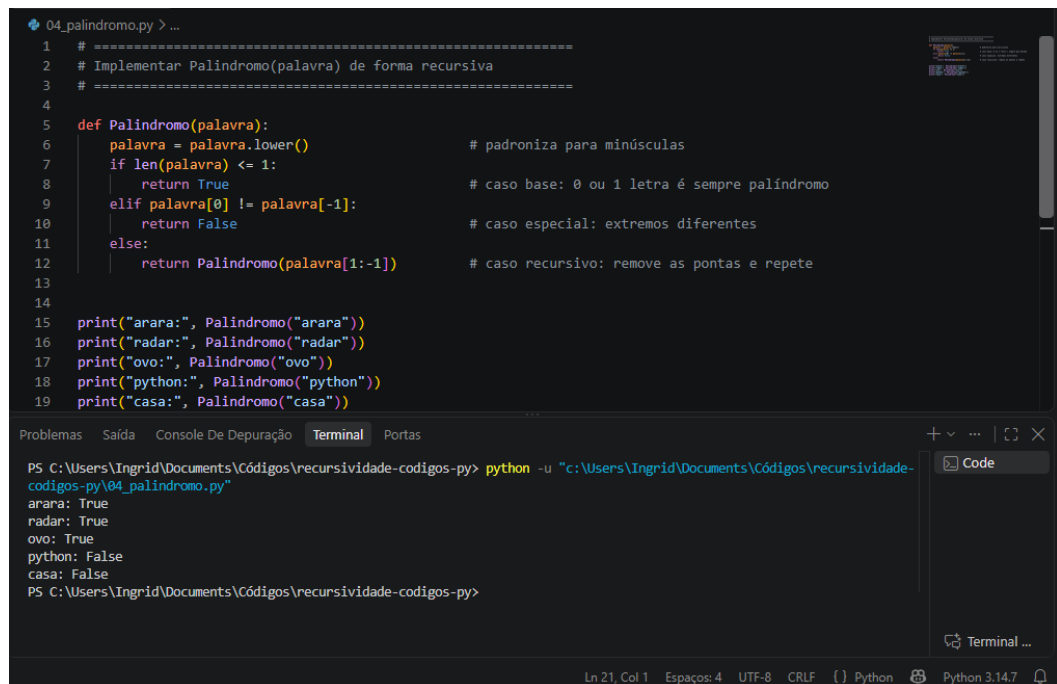
Exemplos de utilização

Utilizei palavras que são palíndromos e outras que não são. Assim, foi possível observar os dois possíveis resultados da função.

```
print("arara:", Palindromo("arara"))
print("radar:", Palindromo("radar"))
print("ovo:", Palindromo("ovo"))
print("python:", Palindromo("python"))
print("casa:", Palindromo("casa"))
```

Resultados obtidos

Após executar o programa, foram obtidos os seguintes resultados:



```
04_palindromo.py > ...
1 # =====
2 # Implementar Palindromo(palavra) de forma recursiva
3 # =====
4
5 def Palindromo(palavra):
6     palavra = palavra.lower()           # padroniza para minúsculas
7     if len(palavra) <= 1:
8         return True                    # caso base: 0 ou 1 letra é sempre palíndromo
9     elif palavra[0] != palavra[-1]:
10        return False                   # caso especial: extremos diferentes
11    else:
12        return Palindromo(palavra[1:-1]) # caso recursivo: remove as pontas e repete
13
14
15 print("arara:", Palindromo("arara"))
16 print("radar:", Palindromo("radar"))
17 print("ovo:", Palindromo("ovo"))
18 print("python:", Palindromo("python"))
19 print("casa:", Palindromo("casa"))

Problemas Saída Console De Depuração Terminal Portas
P5 C:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py> python -u "C:\Users\Ingrid\Documents\Códigos\recursividade-
codigos-py\04_palindromo.py"
arara: True
radar: True
ovo: True
python: False
casa: False
P5 C:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py>

Ln 21, Col 1 Espaços: 4 UTF-8 CRLF () Python Python 3.14.7
```

Print 8 – Execução do Programa Palíndromo

6 FUNÇÃO CONTAOCORRÊNCIA

Nesta questão, a função verifica quantas vezes determinado elemento aparece em um vetor. A cada chamada, ela compara o primeiro elemento com o valor que estamos procurando. Se os valores forem iguais, soma 1 à contagem. Se forem

diferentes, a função segue analisando o resto do vetor. Quando o vetor fica vazio, chegamos ao caso base e a função retorna 0.

```
05_contaOcorrencia.py > ...
2 # Implementar ContaOcorrencia(vet, elemento) de forma recursiva
3 # =====
4
5 def ContaOcorrencia(vet, elemento):
6     if len(vet) == 0:
7         return 0 # caso base: vetor vazio, nenhuma ocorrência
8     elif vet[0] == elemento:
9         return 1 + ContaOcorrencia(vet[1:], elemento) # encontrou: soma 1 e continua no resto
10    else:
11        return ContaOcorrencia(vet[1:], elemento) # não encontrou: continua no resto
12
13
14 print("2 em [1, 2, 3, 2, 4, 2] =", ContaOcorrencia([1, 2, 3, 2, 4, 2], 2))
15 print("5 em [5, 5, 5, 7, 8] =", ContaOcorrencia([5, 5, 5, 7, 8], 5))
16 print("6 em [1, 2, 3, 4, 5] =", ContaOcorrencia([1, 2, 3, 4, 5], 6))
17 print("10 em [10, 20, 10, 30, 10] =", ContaOcorrencia([10, 20, 10, 30, 10], 10))
18
```

Print 9 – Programa ContaOcorrência

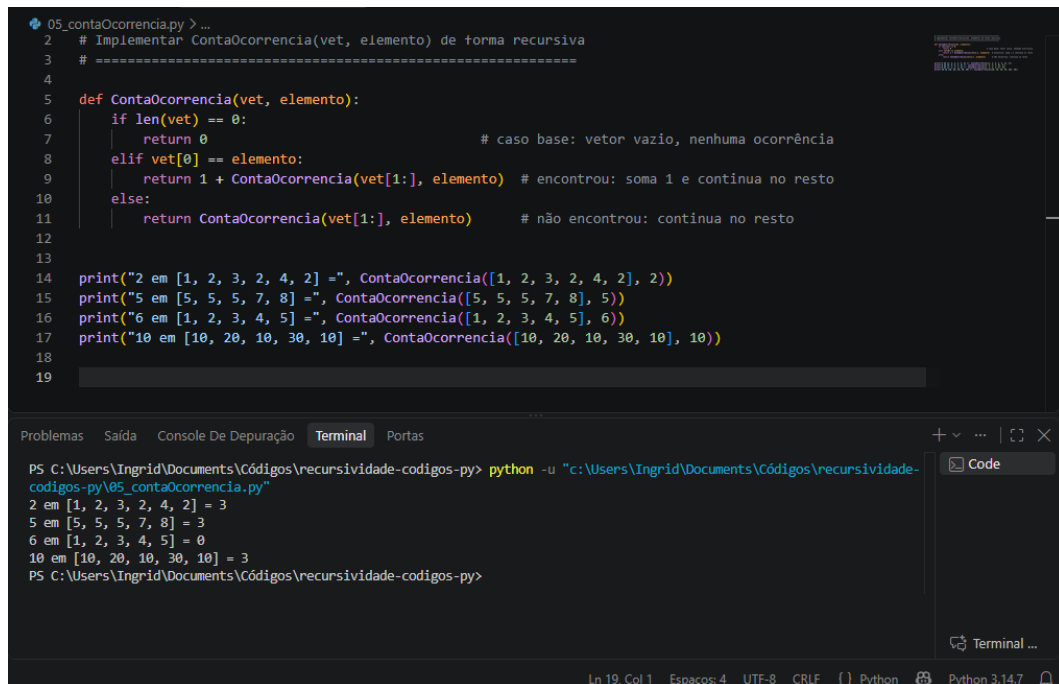
Exemplos de utilização

Nos exemplos, testei situações diferentes: elementos que aparecem várias vezes e também um elemento que não aparece nenhuma vez.

```
print("2 em [1, 2, 3, 2, 4, 2] =", ContaOcorrencia([1, 2, 3, 2, 4, 2], 2))
print("5 em [5, 5, 5, 7, 8] =", ContaOcorrencia([5, 5, 5, 7, 8], 5))
print("6 em [1, 2, 3, 4, 5] =", ContaOcorrencia([1, 2, 3, 4, 5], 6))
print("10 em [10, 20, 10, 30, 10] =", ContaOcorrencia([10, 20, 10, 30, 10], 10))
```

Resultados obtidos

Após executar o programa, foram obtidos os seguintes resultados:



```
05_contaOcorrencia.py > ...
2 # Implementar ContaOcorrencia(vet, elemento) de forma recursiva
3 # =====
4
5 def ContaOcorrencia(vet, elemento):
6     if len(vet) == 0:
7         return 0 # caso base: vetor vazio, nenhuma ocorrência
8     elif vet[0] == elemento:
9         return 1 + ContaOcorrencia(vet[1:], elemento) # encontrou: soma 1 e continua no resto
10    else:
11        return ContaOcorrencia(vet[1:], elemento) # não encontrou: continua no resto
12
13
14 print("2 em [1, 2, 3, 2, 4, 2] =", ContaOcorrencia([1, 2, 3, 2, 4, 2], 2))
15 print("5 em [5, 5, 5, 7, 8] =", ContaOcorrencia([5, 5, 5, 7, 8], 5))
16 print("6 em [1, 2, 3, 4, 5] =", ContaOcorrencia([1, 2, 3, 4, 5], 6))
17 print("10 em [10, 20, 10, 30, 10] =", ContaOcorrencia([10, 20, 10, 30, 10], 10))
18
19
```

Problemas Saída Console De Depuração Terminal Portas

```
PS C:\Users\Ingrid\Documents\recursividade-codigos-py> python -u "c:\Users\Ingrid\Documents\Códigos\recursividade-codigos-py\05_contaOcorrencia.py"
2 em [1, 2, 3, 2, 4, 2] = 3
5 em [5, 5, 5, 7, 8] = 3
6 em [1, 2, 3, 4, 5] = 0
10 em [10, 20, 10, 30, 10] = 3
PS C:\Users\Ingrid\Documents\recursividade-codigos-py>
```

Ln 19, Col 1 Espaços: 4 UTF-8 CRLF {} Python Python 3.14.7

Print 10 – Execução do Programa ContaOcorrência

7 CONCLUSÃO

Com a realização da atividade, ficou claro que a recursividade pode ser usada em problemas bem diferentes. Mesmo que cada questão tenha um objetivo específico, todas seguem uma ideia parecida. Existe um caso mais simples que encerra a função. Enquanto esse caso não é alcançado, a própria função é chamada de novo com um problema menor.

Os exemplos de uso e os testes foram importantes para verificar se cada programa funcionava como esperado. Com eles, foi possível verificar os resultados do fatorial, da sequência de Fibonacci, da soma dos elementos de uma lista, da verificação de palíndromos e da contagem de ocorrências em um vetor. Assim, a atividade ajudou a entender na prática como funcionam as chamadas recursivas.

DISPONIBILIDADE DOS CÓDIGOS

Repositório dos códigos no GitHub:

Os códigos-fonte desenvolvidos nesta atividade estão disponíveis no repositório do GitHub para consulta e verificação:

<https://github.com/IngridRDA/recursividade-codigos-py>

REFERÊNCIAS

CARVALHO, Miguel Gabriel Prazeres de. *Introdução a algoritmos e estrutura de dados em Python*. [S. l.]: Afya, 2024. 32 p. (Percurso de Aprendizagem, Unidade 1).